

Ex.No: 19

Date:

Aim:

TRIGGER

To implement trigger in MySQL.

A trigger or database trigger is a stored program executed automatically to respond to a specific event such as INSERT, UPDATE or DELETE occurring in a table.

Syntax – Create Trigger

```
CREATE TRIGGER trigger_name  
trigger_time trigger_event ON table_name  
FOR EACH ROW  
BEGIN  
    ...  
END;
```

Program 1:

Create a trigger in MySQL to log the changes of the STUDENT table with fields ID, Name and Email. Also create a new table named EMPLOYEES_AUDIT to keep the changes of the employee table. Create a BEFORE UPDATE trigger that is invoked before a change is made to the employees table.

Solution:

```
CREATE DATABASE IF NOT EXISTS TriggerDB;  
USE TriggerDB;
```

```
DROP TABLE IF EXISTS EMPLOYEES_AUDIT;  
DROP TABLE IF EXISTS STUDENT;
```

```
CREATE TABLE STUDENT (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Email VARCHAR(100)  
);
```

```
CREATE TABLE EMPLOYEES_AUDIT (  
    ID INT,
```

```
Name VARCHAR(100),
Email VARCHAR(100),
Action VARCHAR(20),
Change_Date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
INSERT INTO STUDENT (ID, Name, Email)
VALUES
(1, 'Harry Potter', 'harry@gmail.com'),
(2, 'Clark Kent', 'clark@gmail.com'),
(3, 'Tony Stark', 'tony@gmail.com');
```

```
DROP TRIGGER IF EXISTS before_student_update;
```

```
DELIMITER $$
```

```
CREATE TRIGGER before_student_update
BEFORE UPDATE ON STUDENT
FOR EACH ROW
BEGIN
    INSERT INTO EMPLOYEES_AUDIT (ID, Name, Email, Action)
    VALUES (OLD.ID, OLD.Name, OLD.Email, 'BEFORE UPDATE');
END$$
```

```
DELIMITER ;
```

```
UPDATE STUDENT
SET Name = 'Harry James Potter',
    Email = 'harry.james@gmail.com'
WHERE ID = 1;
```

```
SELECT * FROM STUDENT;
```

```
SELECT * FROM EMPLOYEES_AUDIT;
```

Expected Output:

STUDENT table:

1	Harry James Potter	harry.james@gmail.com
2	Clark Kent	clark@gmail.com
3	Tony Stark	tony@gmail.com

EMPLOYEES_AUDIT table:

1 Harry Potter harry@gmail.com BEFORE UPDATE

The screenshot shows a MySQL IDE window with multiple tabs. The active tab, 'SQL File 8*', contains the following SQL script:

```
266 SET Name = 'Harry James Potter',
267     Email = 'harry.james@gmail.com'
268 WHERE ID = 1;
269
270 -- Display updated STUDENT table
271 • SELECT * FROM STUDENT;
272
273 -- Display audit records
274 • SELECT * FROM EMPLOYEES_AUDIT;
```

Below the script, the 'Result Grid' is displayed, showing the data from the 'EMPLOYEES_AUDIT' table. The table has five columns: ID, Name, Email, Action, and Change_Date. It contains one row of data:

ID	Name	Email	Action	Change_Date
1	Harry Potter	harry@gmail.com	BEFORE UPDATE	2026-08-23 23:27:49

The IDE interface includes various toolbars at the top and bottom, and a sidebar on the right with options like 'Result Grid', 'Form Editor', and 'Field Types'.

Result:

Thus, the BEFORE UPDATE trigger was created and executed successfully in MySQL, and the old student details were stored in the EMPLOYEES_AUDIT table.